

DeFa — Ignyte Smart Commerce Infrastructure

Challenge Submission

Polygon Labs — SME Trade Finance Track

Repository: <https://github.com/Mate-Sol/The-Smart-Commerce-Infrastructure-Challenge-Polygon-Labs->

Live demo: <https://defa-polygon-hackathon.invoicemate.net> (access code 123456) **Chain:** Polygon Amoy (chain id 80002)

Team Background

DeFa is built by the InvoiceMate team. Two-person build for this submission:

- **Ibrahim Ansari — CEO.** Trade finance + fintech background. Owns product framing, regulatory framing, and the credit-workflow spec (KAM → CAD → CRO → Legal).
- **Hamza Anjum — CTO.** Full-stack + Solidity. Shipped the entire submission — `payfi_v1` contract set, Node/ethers backend, React lender UI, and the admin/PSP portal.

Backing engineering pod at InvoiceMate provides code review and QA support outside the sprint.

Problem Statement

Cross-border SME trade finance is roughly **\$2 trillion undersupplied globally** (ADB Trade Finance Gaps Report, 2024). Three specific frictions strand working capital in the corridors we target (UAE ↔ Pakistan, UAE ↔ South Africa, Gulf ↔ East Africa):

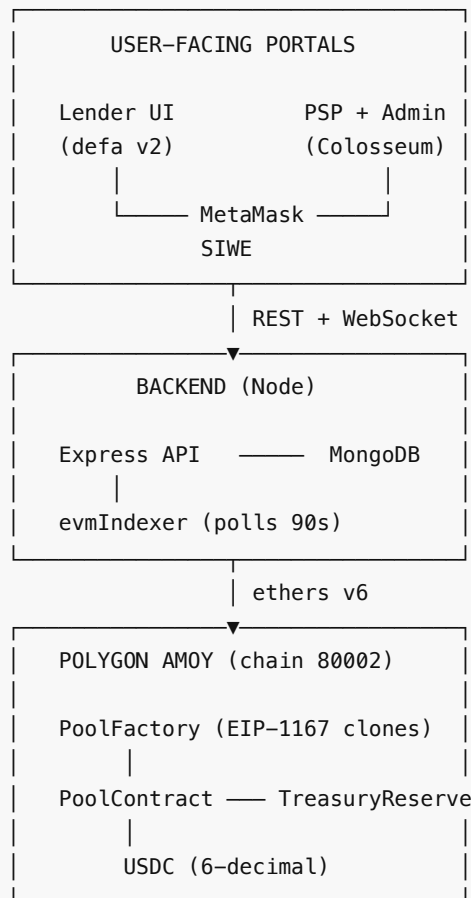
1. **Settlement latency.** A PSP paying a supplier against an invoice financed in another country waits 2–5 business days for SWIFT to clear. The SME is out of stock; the PSP carries the settlement risk on its balance sheet the whole time, and that cost gets priced back into every facility.
2. **FX drag on the drawdown leg.** When a facility is priced in USD, drawn in AED, and settled in PKR, the PSP eats the spread on every drawdown. Multiplied over hundreds of drawdowns per month, 200–400 bps of margin evaporates before it reaches the LP.
3. **No shared credit primitive.** Every bank runs its own KAM → CAD → CRO workflow in a separate silo. A PSP that's been underwritten by one bank has to redo the entire KYB and credit review to onboard with the next. There is no reusable on-chain primitive that says *"this PSP is underwritten, this pool is funded, here's the automated waterfall."*

Why Polygon

Polygon's throughput, sub-cent transaction cost, and EVM tooling maturity make it the natural chain for a credit primitive that has to handle high-frequency drawdown + repayment cycles at SME ticket sizes (\$5k–\$50k typical). At those ticket sizes, mainnet Ethereum gas eats the yield; Polygon leaves it intact.

Technical Architecture

System overview



Full Mermaid diagrams (top-level graph, PSP → KAM → CAD → CRO sequence, contract class) live in [docs/ARCHITECTURE.md](#).

Contract layer — `payfi_v1`

- **PoolFactory** — EIP-1167 minimal-proxy factory. `approvePsp(address)` gates who can operate a pool. `createPool(params)` clones a fresh `PoolContract` initialized with risk-envelope bounds (APR ceiling, utilisation cap, penalty rate, tenor limit).
- **PoolContract (per-facility)** — deposit / withdraw for LPs, `executeDrawdown(ref, receiver, amount, days)` for PSPs, `repay(ref)` waterfall (principal → utilization fee → penalty → protocol split → LP yield), `claimYield` + `claimPrincipal` for LPs, `declareDefault` fallback that draws from `TreasuryReserve`.
- **TreasuryReserve** — protocol fee sink + insurance reserve.
- **MockStablecoin** — 6-decimal USDC surrogate for testnet. Swaps 1:1 to real USDC on mainnet.

Uses OpenZeppelin v5.x (`AccessControl` , `ReentrancyGuard` , `Clones` , `SafeERC20`).
`_disableInitializers` on the implementation contract. WAD fixed-point math with invariant tests. ~17K LOC of Foundry test coverage across 26 test files (unit, invariant, adversarial, differential, fuzz, gas-profile).

Backend layer — Node / Express / ethers v6

- 18 route files: auth (SIWE + JWT), pool discovery, facility lifecycle, PSP KYB, KAM/CAD/CRO/Legal/on-chain-admin approvals, faucet, admin build-tx endpoints.
- `poolServiceEvm` — reads live pool state and builds calldata for user wallets to sign.

- `walletAuthEvm` — SIWE (EIP-4361) with nonce store and domain binding.
- `evmIndexer` worker — polls `PoolCreated`, `Deposit`, `DrawdownExecuted`, `Repaid` on a 90s window and mirrors state into Mongo.
- `_withRetry` on all RPC calls — coalesces transient RPC errors under burst load.

Frontend layer — two portals

- **Lender UI** (`client/`) — React 19 + Vite + Tailwind v4 + wagmi v2 + viem v2 + RainbowKit v2.
Browse pools, deposit USDC (2-step approve + deposit), watch utilisation live, redeem principal + yield.
- **PSP + admin portal** (`client-legacy/`) — PSPs submit KYB and request facilities;
KAM/CAD/CRO/Legal walk each facility through approvals; on-chain admin signs the two-transaction pool bootstrap (`approvePsp` + `createPool`).

End-to-end lifecycle

- | | |
|-------------------------------------|---|
| 1. PSP signs up | → KYB submission created |
| 2. PSP requests facility | → <code>Facility.status = KAM_REVIEW</code> |
| 3. KAM approves | → → <code>CAD_REVIEW</code> |
| 4. CAD approves | → → <code>CRO_REVIEW</code> |
| 5. CRO approves + terms | → → <code>AWAITING_POOL_INIT</code> |
| 6. Onchain admin signs | → <code>factory.approvePsp</code> + <code>factory.createPool</code> |
| 7. <code>evmIndexer</code> picks up | → pool visible on <code>/pools</code> within 90s |
| 8. LP deposits | → approve + deposit |
| 9. PSP executes drawdown | → server signs <code>AGENT2</code> , USDC to receiver |
| 10. PSP repays | → principal + utilisation fee + penalty |
| 11. LP redeems | → <code>claimYield</code> + <code>claimPrincipal</code> |

Deployed on Polygon Amoy (chain 80002)

Contract	Address
PoolFactory	0xE02D8d3B14746E42c5D41a2CA805798D5A6E0F78
MockUSD (6-decimal USDC surrogate)	0x4e39880B43f9a83586a2aC75a01dff779Eb958c0
TreasuryReserve	0x03D21aFF05a94E2B87d1F55b2deE8F6f3fd9D9f8

Three demo facilities are live on-chain right now: Mercury Settlements USDC Facility (12% APR, medium risk), Aurum Cross-Border Corridor (6% APR, low risk), Meridian FX Working Capital (14% APR, medium risk).

Test coverage — 38 integration tests, all green on real Polygon Amoy

Suite	Result
<code>e2eFlows.test.js</code> — auth, marketplace, deposit build-tx, faucet, gates	18/18
<code>lifecycleFlows.test.js</code> — PSP → KAM → CAD → CRO → onchain admin gating	13/13
<code>onchainFlows.test.js</code> — real <code>approvePsp</code> + <code>createPool</code> + faucet mint + BE state read	7/7

Same suite runs green on Arc Testnet in the sibling repo — 76 total integration tests across the two chains.

Launch Roadmap

Phase 1 — Sprint (through Aug 2026)

- **Jul 13** — Ignyte submission (this).
- **Jul–Aug** — Pilot facility with a PSP partner in the UAE ↔ Pakistan corridor.
- **Aug** — Engage third-party audit for the `payfi_v1` contract set.

Phase 2 — Q4 2026

- Migrate `MockStablecoin` → native USDC on Polygon PoS mainnet.
- `PoolFactory v2` — configurable per-pool risk envelopes, protocol-fee auction, multi-asset support.
- Legal wrapper for pool holders (SPV or Prescribed Company) with LP-share tokenisation.

Phase 3 — H1 2027

- Three live facilities across UAE ↔ Pakistan, UAE ↔ South Africa, and Gulf ↔ Kenya corridors.
- Target: **\$10M cumulative drawdown volume** across pilot facilities.
- Agent-driven KAM / CAD reviews live in production behind a human-approval fallback.

Phase 4 — H2 2027

- Open `payfi_v1` to third-party PSPs — any PSP can spin up a pool without running the underwriting stack themselves.
- Multi-chain deployment (Polygon + Arc + Base) with CCTP-based liquidity balancing.
- Target: **\$50M cumulative drawdown volume**.